Ingeniería de Plataforma

# Implementación de OCP Multicluster

**DEFINICIÓN DE ESTRATEGIA Y TOPOLOGÍA GENERAL DE COMPONENTES**

**Versión 1.0**

**Banco Galicia**

*Documento Confidencial*



Galicia

# Índice

**Estructura general** 3

Objetivo del documento 3
Alcance y Objetivo 3
Resumen ejecutivo 3

**Situación actual** 4

Limitaciones técnicas prioritarias 4
Requerimientos Específicos 12
Definiciones técnicas y operativas 14

**Estrategia de evolución general** 15

Lecciones sobre el análisis 10
Fases de implementación 12
Topología target 14

**Plan de ejecución** 16

# Estructura General

## OBJETIVO DEL DOCUMENTO

El objetivo del presente documento es describir la situación actual del Banco Galicia en el uso y operación de la plataforma OpenShift. Mediante este análisis se identifican una serie de definiciones relacionadas a la evolución de esta tecnología y soportadas sobre las pruebas realizadas por el equipo en la investigación y desarrollo de nuevos paradigmas de uso y ejecución de microservicios. Como resultado final se detalla un plan de alto nivel de ejecución de iniciativas que permitirán alcanzar el objetivo final.

**Comentado [DD1]:** Hagamos foco en atacar los riesgos la plataforma y escalabilidad a futuro

## ALCANCE Y OBJETIVO

La reingeniería de nuestra plataforma OpenShift (OCP) tiene como objetivo rediseñar, modernizar y optimizar la arquitectura, los componentes y los procesos operativos que sostienen la estructura de contenedores del Banco Galicia. Esta iniciativa busca asegurar la escalabilidad futura para cubrir las necesidades de la organización, soportando el crecimiento de cargas críticas, los modelos multicluster y los requisitos de disponibilidad y resiliencia exigidos por la industria financiera.

El objetivo principal es evolucionar la plataforma desde su estado actual hacia un modelo que garantice 7 aspectos:

**Comentado [DD2]:** Hay que incorporar lo del blast rad acá, es una de las razones principales

- Mayor eficiencia operativa mediante automatización, estandarización y uso de un stack de tecnologías que construyan un framework.
- Escalabilidad y elasticidad para soportar múltiples dominios de negocio y picos de transaccionalidad/demanda.
- Alta disponibilidad y resiliencia mediante topologías multicluster y prácticas estándares de mercado que minimicen el impacto cruzado de fallas.
- Reducir la complejidad técnica y eliminar componentes legacy sin evolución y con proyección de tecnologías fuera de soporte.
- Mejorar la experiencia del desarrollo habilitando esquemas de seguridad y comunicación más controlados que permitan darle mejor trazabilidad al consumo de servicios internos como externos.
- Fortalecer la postura de ciberseguridad bajo estándares de la industria bancaria.
- Implementar capacidades de observabilidad integral para una operación más proactiva.

## RESUMEN EJECUTIVO

**Comentado [DD3]:** No pondria resumen ejecutivo aca, quedó como en el medio de la narrativa, terminemos el c y hacemos un resumen de 1 o 2 paginas al inicio.

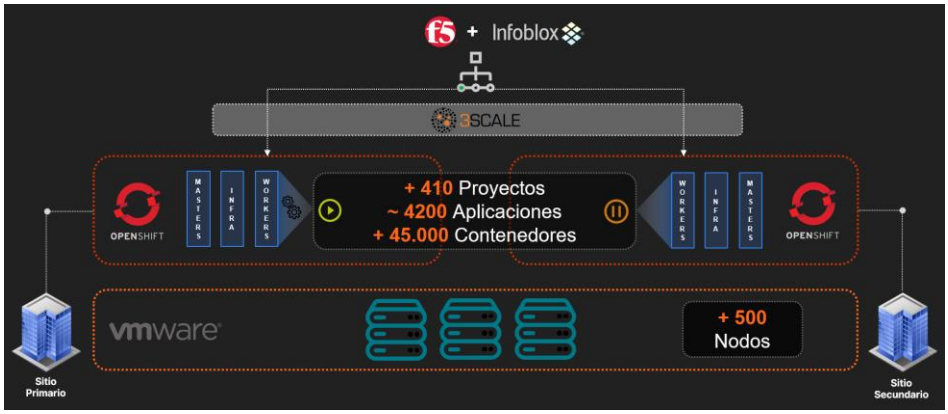
La transformación prioritaria de plataforma no es solo un reemplazo puntual de APIM por fin de vida del producto, sino la evolución desde un esquema monolítico hacia una arquitectura multicluster con gobierno central y operación desacoplada por dominios. Nuestro API Manager se utiliza como caso modelador para validar decisiones de red, seguridad, resiliencia y operación, que luego se extienden al resto de OpenShift.

La situación actual representa un alto riesgo sistémico: un cluster productivo de gran escala soportando cargas críticas de múltiples líneas de negocio, con alto volumen transaccional y fuerte dependencia de procesos manuales para red, DNS, certificados, sincronización intersite y continuidad. Este diseño amplifica blast radius, extiende ventanas de mantenimiento y limita la elasticidad real.

La estrategia objetivo define una colección de clusters con responsabilidades claras, segmentación por criticidad y tipo de servicio, patrones diferenciados para tráfico north-south y east-west, y un modelo operativo GitOps + IaC para ciclo day 0/day 1/day 2. El beneficio general esperado es reducir el impacto cruzado, mejorar continuidad operativa y sostener crecimiento sin incrementar proporcionalmente la complejidad.

## Situación actual

### Topología y capacidad (As-is)

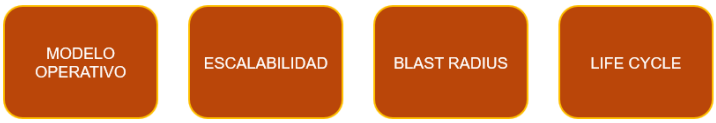



- ✓ Cluster OpenShift monolítico de referencia: mas de 500 nodos, mas de 45.000 pods y mas de 500 namespace, distribuidos sobre 4 ambientes, siendo el productivo un sistema de cluster activo-pasivo establecido por el balanceo de la comunicación.
- ✓ Volumen transaccional aproximado: 8 mil millones de requests/mes, con predominio east-west (7,5B) sobre north-south (~500M).
- ✓ Topología stretch entre Plaza (PGA) y Matriz (CMZ), con contingencia APIM en esquema activo-standby.
- ✓ Alta densidad de exposición en capa de ingreso y APIM: reportes de múltiples ingress controllers y ~2.200 APIs productivas.

**Comentado [DD4]:** Ojo, asi descrito parece 1 cluster c todos los ambientes, pondria foco en el de prod y en tod caso aclaramos a cuanto asciende el total de pods y nodo tomando todos los clusters

### Limitaciones técnicas prioritarias

Existen múltiples limitaciones técnicas que colisionan con las necesidades de crecimiento de Banco Galicia, pudiéndose agrupar en las siguientes 4 categorias:



**MODELO OPERATIVO**

- Alta manualidad en tareas recurrentes: alta/modificación de VIPs, DNS, certificados, sincronización entre sitios, DR y cambios de red.
- Dependencia de tickets y coordinación interequipos para cambios críticos, con variabilidad en tiempos de ejecución.
- Coexistencia de automatización parcial con procedimientos manuales en procesos de alto impacto.

**ESCALABILIDAD**

- Unidad principal de escalado concentrada en un cluster monolítico, con overcommit reportado en partes del entorno.
- Limitaciones operativas de APIM actual para crecimiento de rutas/APIs y recargas no dinámicas.
- Hair-pinning en tráfico interno que agrega latencia y penaliza escalabilidad transaccional.

**IMPACTO CRUZADO (BLAST RADIUS)**

**Comentado [DD5]:** Acá hay que hablar de incidentes

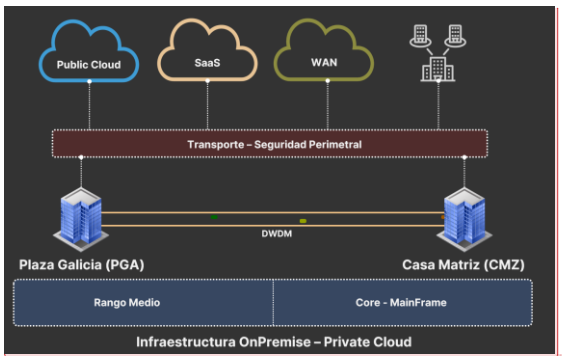
- La falla de capacidad, red, storage o configuración puede propagarse en forma transversal a múltiples dominios.
- Dependencias compartidas (balanceo, DNS, storage, identidad, APIM) elevan el impacto simultáneo en canales e integraciones.
- Mayor complejidad de contención durante incidentes por concentración de cargas críticas en la misma base de plataforma.

**MANTENIMIENTO, LIFECYCLE Y VENTANAS**

- Ventanas de mantenimiento extensas por tamaño de cluster y secuencias de actualización por etapas.
- Riesgo de deriva de configuración intersitio cuando la sincronización no es completamente declarativa.
- Presión de ciclo de vida: OpenShift en rango 4.15-4.16 y objetivo de evolución a 4.20.x; 3scale con EOL en 2027.

## Diagrama topológico de referencia actual

**DATACENTERS**




**Comentado [DD6]:** Veamos si tenemos algo que deje r claro como están "apoyados" hoy los clusters en los DC

- Dos sitios principales (Plaza y Matriz) con red extendida mediante un sistema DWDM de <Chequear>40 Gbps de ancho de banda.
- Capa perimetral con F5/Fortinet y componentes de seguridad de borde, tanto para la creación de túneles seguros con distintos proveedores, tráfico saliente hacia Internet y la publicación de aplicaciones via APIs.

**FLUJOS**

- North-south: Internet-partners(DMZ)/core-legacy -> Ingress Sharding OpenShift/APIM -> servicios.
- East-west: en varios recorridos internos el tráfico sale y reingresa por balanceadores externos para resolver autorización/ruteo.

**Con formato:** Inglés (Estados Unidos)

**HARDWARE Y COMPONENTES CRÍTICOS**

- Dependencias de storage compartido y componentes transversales con efecto banco-wide ante falla. 7
- Integración fuerte con componentes de red corporativa (DNS, balanceo, certificados). 5

**VERSIONES Y EVOLUCIÓN**

- Base actual en 4.15-4.16, con objetivo de evolución de plataforma y CNI hacia 4.20.x. 42
- Dependencias de compatibilidad (incluyendo storage) condicionan secuencia de upgrade. 7

**Comentado [DD7]:** No está todo en 4-16?

# 3. Arquitectura objetivo multicluster

## 3.1 Segmentación de dominios y tipología de clusters

El modelo objetivo propone segmentar por criticidad y función para reducir blast radius y desacoplar crecimiento:

- Clusters de negocio para cargas aplicativas distribuidos por tribu/dominio y tomando como referencia el uso de recursos por cada uno de ellos (vCPU, RAM, Q Apis). Si bien esta es una primera segmentación, a futuro este~~a~~ agrupamiento puede modificarse en relación al crecimiento de los proyectos que componen a cada tribu/dominio

**Comentado [DD8]:** No estoy de acuerdo con esto, charlemoslo, creo que acá lo crítico es separar canales de productos y servicios cross

| Grupo 1 | Grupo 2 | Grupo 3 | Grupo 4 | Grupo 5 | Grupo 6 |
|---|---|---|---|---|---|
| • Arquitectura de Canales | • Coe Automation<br>• Tribu Every Day Banking<br>• Tribu Inversiones<br>• Tribu Medios de Pago | • Administración & Finanzas<br>• Pagos y Transferencias Core<br>• Tribu Cobros y Pagos | • Core Banking<br>• Sucursales y Canales<br>• Tribu Contactabilidad<br>• Tribu Omnicanalidad | • Arquitectura<br>• Desarrollo<br>• DevSecOps<br>• Gestión Comercial<br>• Indeterminada<br>• QA<br>• Stand Alone<br>• Tecnología<br>• Tribu Comex<br>• Tribu Data & Analytics<br>• Tribu Loyalty<br>• Tribu Segmentos | • Inversiones Core<br>• Tribu Comercios<br>• Tribu Lending |

- Cluster de servicios comunes para capacidades transversales (observabilidad, secretos, herramientas de plataforma).

>>>DIAGRAMA

- Clusters de gestión y gobierno para operación multicluster, políticas globales y ciclo de vida.

>>>>DIAGRAMA

- Separación explícita entre servicios críticos y no críticos.

Como referencia de escala, se plantea evolución desde el cluster monolítico hacia una flota cercana a 30 clusters totales, con 7-8 productivos según madurez del programa. 34

**Comentado [DD9]:** Empecemos con menos no mas de 5

## 3.2 Modelo de control plane y data plane

- Gobierno central multicluster en topología hub-spoke (por ejemplo ACM), con políticas y ciclo de vida controlados desde repositorios versionados. 40
- Data planes distribuidos por cluster/sitio con autonomía operativa ante pérdida temporal de conectividad al control plane. 40
- Distribución de GitOps por dominios funcionales (infra, seguridad/RBAC, aplicaciones, middleware/APIs) para escalar ownership sin perder control. 40

## 3.3 Patrones de tráfico objetivo

### *North-south (ingreso y exposición)*

- Arquitectura de tres capas: perímetro (DMZ), API Gateway para gobierno L7 y capa de servicios internos. 35
- Considera como north-south también consumo desde legacy/core hacia APIs alojadas en OpenShift. 35
- Capacidades esperadas: OAuth2/JWT, mTLS, rate limiting, cuotas, versionado, telemetría transaccional y auditoría. 35

**Comentado [DD10]:** No tengo acceso a ver el diagram pondría algo acá esquemático, vamos a tener 1 cluster D solamente, no?

### *East-west (comunicación interna)*

- Malla sidecarless para comunicación interna e intercluster. 36
- Decisión vigente: Cilium Mesh para dominio east-west multicluster. 36
- Beneficios esperados: eliminación de hair-pinning, menor latencia, menor complejidad de red, mejor troubleshooting y mayor aislamiento de fallas. 36
- Condición de adopción: pruebas obligatorias de pod churn cross-cluster antes de go-live. 36

**Comentado [DD11]:** Creo que no es el momento para hacer esta movida, mantendría la filosofía N/S para todo inter-cluster e intra-cluster

## 3.4 Ingress/egress, DNS global y balanceo

- Ingreso estandarizado por entorno y dominio; egress con políticas explícitas y trazabilidad. 37
- DNS global con health checks para distribución de tráfico entre clusters. 37
- Automatización de actualizaciones DNS (por ejemplo external-dns) e integración con balanceo global (F5/Infoblox). 37
- Objetivo operativo: conmutación rápida entre sitios con mínima intervención manual. 37

**Comentado [DD12]:** Iríamos a activo-activo? Revisemo esto

### 3.5 Seguridad integral multicluster

- IAM integrado con identidad corporativa y principio de mínimo privilegio. 38
- RBAC declarativo por repositorio, reconciliación continua y segregación de funciones. 38
- Migración desde credenciales estáticas hacia mecanismos con expiración y revocación (OAuth2/JWT + mTLS). 38
- Vault como backend de secretos con sincronización controlada en Kubernetes. 38
- Políticas de red por defecto deny, controles explícitos de comunicación y egress. 38

### 3.6 Observabilidad federada y confiabilidad

- Federación de métricas, logs y trazas para lectura unificada cross-cluster. 39
- OpenTelemetry como patrón transversal y eBPF para visibilidad de red/mesh y mapeo de dependencias reales. 39
- Definición de SLI/SLO por servicio y dominio, con umbrales y ownership explícitos. 39
- Integración de alertado, incident response y postmortems en una misma disciplina operativa. 39

### 3.7 Modelo operativo objetivo

- GitOps + IaC como estándar de cambio para infraestructura, políticas y aplicaciones. 40
- Automatización day 0/day 1/day 2 para reducir tareas manuales recurrentes y drift. 40
- Self-service con templates de plataforma para provisión de entornos, onboarding técnico y despliegue con guardrails. 49
- Operating model de plataforma-producto con roles claros: Platform Engineering, Seguridad, Redes/Comunicaciones, SRE/DevOps y equipos de producto. 48

## 4. Decisiones arquitectónicas más relevantes (problema -> decisión -> beneficio)

| Problema estructural | Decisión arquitectónica | Beneficio esperado |
|---|---|---|
| Concentración de riesgo en cluster único | Segmentación multicluster por dominio/criticidad | Reducción de blast radius y mejor continuidad [34][20] |
| Hair-pinning y latencia en tráfico interno | Malla sidecarless east-west (Cilium Mesh) | Menor latencia y menor dependencia de red legacy [36][27] |
| Mezcla de necesidades externas e internas en APIM | Separación north-south (API Gateway) vs east-west (mesh) | Gobierno L7 donde aporta valor, eficiencia en tráfico interno [35][26] |
| Manualidad operativa y drift entre sitios | GitOps + IaC + control multicluster centralizado | Cambios auditables, repetibles y con rollback [40][18] |

**Comentado [DD13]:** Que de todo esto es mandatorio/urgente y que es para desarrollar durante el año?

**Comentado [DD14]:** Esto no lo veo como un requisito esta iniciativa, en todo caso es del SRE

**Comentado [DD15]:** Veamos cuanto ahora y cuanto en iteraciones posteriores

**Comentado [DD16]:** Revisemos en general que es urge y que se puede ir sumando después

**Con formato:** Inglés (Estados Unidos)

**Con formato:** Inglés (Estados Unidos)

| | | |
|---|---|---|
| Seguridad basada en credenciales estáticas | Identidad de workload, RBAC declarativo, Vault, mTLS | Mejor cumplimiento, revocación efectiva y trazabilidad [38][54][55] |
| Observabilidad fragmentada | Observabilidad federada con OTel + eBPF | Diagnóstico end-to-end y reducción de MTTR [39][29][60] |
| DR con alta intervención manual | DNS global + health checks + runbooks/drills + automatización progresiva | Mejor RTO efectivo y menor variabilidad operativa [41][37] |

**Con formato:** Inglés (Estados Unidos)

# 5. Estrategia de evolución multicluster (detalle por fases)

## 5.1 Fase fundacional: habilitadores de plataforma

Objetivo: establecer base común para ejecutar la migración sin incrementar riesgo. 44

- Definir baseline de seguridad, observabilidad y gobierno técnico por cluster. 18
- Formalizar repositorio de verdad para RBAC, políticas de red, configuración de ingreso/egreso y secretos. 40
- Alinear modelo day 0/day 1/day 2 con ownership y RACI por dominio. 48
- Preparar estrategia de despligue, versiones y dependencias para ruta de upgrade OCP hacia 4.20.x. 46

**Con formato:** Inglés (Estados Unidos)

**Código de campo cambiado**

## 5.2 Fase de sharding de ingreso y desacople de flujos

Objetivo: desacoplar rutas y permitir transición gradual sin cortes masivos. 35

- Separar puntos de ingreso por función durante transición (gestión OCP, rutas actuales, gateway interno, API management externo). 35
- Habilitar migración selectiva con VIPs/CNAMEs por proyecto, evitando switcheo total. 35
- Mantener coexistencia controlada entre modelo actual y destino, con validación automática para evitar drift. 40

## 5.3 Fase de segmentación operativa y gobierno

Objetivo: pasar de operación centralizada por excepción a operación por dominios con guardrails. 34

- Activar control plane central multicluster con data planes distribuidos. 34
- Estandarizar APIM/API Gateway como capacidad north-south multitenant por necesidad de dominio. 35
- Automatizar RBAC y políticas globales con reconciliación continua. 54
- Ejecutar upgrades por etapas (control plane/componentes core y luego pools de cómputo), con pruebas de riesgo sobre migraciones de red/CNI. 46
- Fortalecer continuidad con ejercicios DRP periódicos, runbooks y criterios de no-go-live. 41

## 5.4 Fase de movimiento de proyectos y cargas

Objetivo: redistribuir capacidad y reducir presión sobre el monolito sin refactor funcional prematuro. 44

- Identificar aplicaciones HA-ready como primera ola de migración. 44
- Aplicar enfoque lift-and-reshape por dominios y oleadas, con coexistencia temporal origen/destino. 44
- Reorganizar namespaces y cargas por criticidad/función para aislar impacto. 34
- Estandarizar templates de aplicación y pipelines de movimiento automatizado. 53
- Ejecutar switcheo progresivo de tráfico con rollback controlado. 44

### 5.5 Fase de consolidación de patrones de alta disponibilidad

Objetivo: estabilizar operación multicluster con capacidad de recuperación verificable. 41

- Adoptar modelos activo-activo o activo-pasivo según criticidad y naturaleza del servicio. 41
- Integrar DNS global, balanceo y health checks multicapa para conmutación controlada. 37
- Definir RTO/RPO por dominio de negocio y validar con drills representativos. 41
- Asegurar recuperación de estado y datos para cargas stateful, no solo redeploy de manifiestos. 41

## 6. Topología target consolidada

### 6.1 Vista lógica

- Capa de gobierno multicluster (control plane) para políticas y ciclo de vida. 34
- Capa de ejecución distribuida (data planes) en clusters por dominio. 34
- Capa de ingreso north-south para exposición externa y canales/core/legacy. 35
- Capa de comunicación east-west para tráfico interno/intercluster. 36

### 6.2 Roles por tipo de cluster

- **Clusters de negocio:** ejecución de servicios de dominio con SLO/SLA propios. 34
- **Clusters de servicios comunes:** observabilidad, secretos, componentes compartidos. 39
- **Clusters de gestión:** gobierno de flota, GitOps/IaC, políticas globales. 40
- **Clusters especializados:** casos de uso específicos con requisitos técnicos particulares. 34

## 7. Plan de ejecución high-level

### 7.1 Horizonte 2026-2027

- **Q1 2026:** cierre de definiciones de arquitectura objetivo y gobierno de ejecución. 1
- **Q2 2026:** implementación de habilitadores multicluster, POC de hardening/performance y validación de escenarios críticos. 4433
- **Q3-Q4 2026:** migración progresiva por oleadas (dominios, namespaces, criticidad), con pilotos y expansión controlada. 44
- **Q4 2026:** consolidación operativa de la nueva topología para dominios priorizados. 34
- **2027:** cierre de transición del dominio APIM antes de EOL de 3scale. 6

**Comentado [DD17]:** Hacer esto en 2027 vale 500 luca
verdes, el plan es hacerlo este año.

### 7.2 Entregables de control ejecutivo

- Arquitectura detallada aprobada por dominio. 1
- Matriz de dependencias y secuenciamiento técnico. 44
- Criterios de avance/no-go-live por fase. 41
- Tablero de salud técnica por cluster y dominio. 63
- Plan integrado de riesgos, mitigaciones y contingencia. 14

# 8. Riesgos críticos y mitigaciones del programa multicluster

**Riesgo 1: inestabilidad en patrones cross-cluster críticos**

Mitigación: pruebas obligatorias de pod churn cross-cluster en POC/staging/preproducción, más criterio de no-go-live si no hay estabilidad consistente. 36

**Riesgo 2: complejidad de upgrade de plataforma y red**

Mitigación: ejecución por etapas, validaciones técnicas previas por dominio, gestión explícita de dependencias de compatibilidad. 46

**Riesgo 3: deriva de configuración entre clusters/sitios**

Mitigación: baseline declarativo, reconciliación continua por GitOps y controles de drift como parte del monitoreo operativo. 40

**Riesgo 4: sobrecarga operativa durante coexistencia de modelos**

Mitigación: migración por oleadas, scope limitado por fase, automatización de tareas repetitivas y reforzamiento de runbooks. 44

**Riesgo 5: brechas de seguridad en transición de identidades/secretos**

Mitigación: plan de migración por dominio, separación de funciones, trazabilidad de cambios y eliminación gradual de credenciales estáticas. 3855

**Riesgo 6: brechas de observabilidad en operación federada**

Mitigación: instrumentación por defecto en clusters nuevos, catálogo único de indicadores y correlación de señales de aplicación/plataforma/red. 3960

# 9. Conclusión

La transformación multicluster es la decisión estructural central para sostener continuidad bancaria, escalar capacidades y reducir riesgo sistémico. El reemplazo de APIM es un frente relevante dentro de esa transformación, pero no su objetivo final. 34

El éxito depende de ejecutar en secuencia: segmentación por dominios, automatización operativa end-to-end, gobierno técnico central con autonomía de data planes, y validación estricta de resiliencia en escenarios reales. Este enfoque permite pasar de una plataforma concentrada y reactiva a una arquitectura distribuida, auditable y preparada para crecimiento sostenido. 4044

# 10. Referencias a documentación detallada (@arquitectura)

## 10.1 Índice y visión general


- 1 Índice estratégico de reingeniería
- 2 Visión y estrategia multicluster


## 10.2 Estado actual y diagnóstico (as-is)


- 3 3.1 Topología actual y capacidad instalada
- 4 3.2 Modelo operativo (día 0, día 1, día 2)
- 5 3.3 Networking, ingress/egress y exposición de servicios
- 6 3.4 Gestión de APIs y estado de APIM
- 7 3.5 Almacenamiento y servicios de datos
- 8 3.6 Seguridad actual (IAM/RBAC, secretos, cifrado, políticas)
- 9 3.7 Observabilidad y monitoreo actual
- 10 3.8 Costos operativos y de licenciamiento actuales
- 11 4.1 Riesgo sistémico y blast radius
- 12 4.2 Límites de escalabilidad y elasticidad
- 13 4.3 Complejidad operativa y tareas manuales
- 14 4.4 Brechas de resiliencia y recuperación ante desastres
- 15 4.5 Brechas de observabilidad y trazabilidad end-to-end
- 16 4.6 Brechas de seguridad y gobierno técnico
- 17 4.7 Complejidad heredada y fricción para equipos


## 10.3 Arquitectura objetivo y decisiones


- 18 5.1 Estandarización y automatización por defecto
- 19 5.2 Escalabilidad horizontal y elasticidad
- 20 5.3 Resiliencia multicluster y alta disponibilidad
- 21 5.4 Seguridad by design y zero trust
- 22 5.5 Observabilidad integral y operabilidad
- 23 5.6 Portabilidad, desacople y minimización de vendor lock-in
- 24 5.7 Simplicidad operativa y reducción de complejidad técnica
- 25 6.1 Marco de evaluación y criterios comparativos
- 26 6.2 Alternativas de API Management y API Gateway
- 27 6.3 Alternativas de service mesh para tráfico este-oeste
- 28 6.4 Alternativas de networking y service discovery
- 29 6.5 Alternativas de observabilidad (métricas, logs, trazas, eBPF)
- 30 6.6 Alternativas de gestión de secretos e identidad
- 31 6.7 Alternativas de operación multicluster y gobierno de flota
- 32 6.8 Evaluación de trade-offs técnicos/operativos/económicos/riesgo
- 33 6.9 Recomendación tecnológica por dominio
- 34 7.1 Modelo multicluster objetivo y segmentación de dominios

- 35 7.2 Patrón norte-sur (ingreso, exposición y gobierno de APIs)
- 36 7.3 Patrón este-oeste (malla y seguridad de comunicación)
- 37 7.4 Arquitectura de ingress/egress y DNS global
- 38 7.5 Modelo de seguridad integral
- 39 7.6 Observabilidad federada multicluster
- 40 7.7 Modelo operativo GitOps + IaC
- 41 7.8 Patrones de resiliencia, failover y continuidad


### 10.4 Evolución, operación y ejecución


- 42 8.1 Estrategia híbrida on-premise + cloud
- 43 8.2 Criterios de elegibilidad y priorización de workloads
- 44 8.3 Enfoque de migración progresiva con mínimo refactor
- 45 8.4 Interoperabilidad entre plataformas y entornos
- 46 8.5 Dependencias críticas para adopción cloud
- 47 8.6 Estrategia de salida y reemplazabilidad tecnológica
- 48 9.1 Operating model de plataforma
- 49 9.2 Self-service y automatización de provision
- 50 9.3 Framework tecnológico estandarizado para equipos
- 51 9.4 Prácticas de entrega segura (CI/CD y gobernanza)
- 52 9.5 Gestión de capacidad, SLO/SLA y operación continua
- 53 9.6 Mejora de developer experience y productividad


### 10.5 Seguridad, cumplimiento y observabilidad


- 54 10.1 Gobierno de identidades y accesos multicluster
- 55 10.2 Gestión de secretos y credenciales
- 56 10.3 Hardening de plataforma y seguridad de red
- 57 10.4 Trazabilidad, auditoría y evidencias regulatorias
- 58 10.5 Gestión de vulnerabilidades y seguridad de cadena de suministro
- 59 11.1 Arquitectura de telemetría (métricas, logs, trazas, eventos)
- 60 11.2 Observabilidad de red y servicios (incluyendo eBPF)
- 61 11.3 Monitoreo de experiencia de aplicación y dependencias
- 62 11.4 Alertado, respuesta a incidentes y postmortems
- 63 11.5 Indicadores de salud técnica por cluster y dominio